Name: ANSWER KEY

Mid-Term Exam Solutions

ECE 251 — Computer Architecture
Spring 2026

12 March 2026
Prof. Rob Marano

Take-Home Exam Instructions & Scheduling:

- **Scheduling:** Email and direct message me on Teams by **Friday, March 13 (tomorrow)** to lock in your testing slot. The testing window is from March 13 through **Wednesday, March 25**.
- **Taking the Exam:** At your scheduled time, I will send the PDF. You have exactly **90 minutes** (approved accommodations apply) to complete 6 problems (100 pts), closed-book, using only the attached MIPS Green Sheet. Provide clear, brief solutions with explicit assumptions. Show all work and **box** final answers.
- **Completing & Submitting:** Print the exam, complete your work, and use a scanner to create a **single OCR-compatible PDF**. Return this PDF via email reply or Teams before your timer expires. Recommended free scanner apps: Apple Notes, Google Drive, Adobe Scan, Microsoft Lens, or Genius Scan.
- **Academic Integrity:** As stated in the Academic Standards, you must produce original work. By signing this exam, you affirm to abide by the Honor Code. Plagiarism of any kind is prohibited.

Exam Topics

The following topics were presented and reviewed in our class. All topics will be covered by the examination problem set that follows.

- Computer abstraction and technology
  - Basic components of a computer
  - Stored Program concept
  - Performance of a computer
  - Big- and Little-Endian memory
- Instructions as the language of the computer
  - Design principles for creating computer architecture
  - Operations & operands of computer hardware
  - Representing instructions in the computer
  - MIPS ISA referencing immediate values and memory addresses
  - Three (3) types of MIPS instructions
  - Converting a line of MIPS assembly code to machine code
  - Procedures, stack pointer (`$sp`), and return address (`$ra`)

1. (*15 points*) CPU Performance & Clock Cycles

Our favorite program runs in 25 seconds on Computer A, which has a 2 GHz clock rate. We are trying to help a computer designer build Computer B, which will run this program in exactly 5 seconds. The hardware designer has determined that a substantial increase in the clock rate is possible, but this increase will natively affect the rest of the physical CPU design, causing Computer B to require 1.2 times as many hardware clock cycles as Computer A for this same program.

Determine the exact clock rate (in GHz) we should tell the designer to target for Computer B. Show all mathematical derivations.

Solution:

1. Calculate the raw number of structural clock cycles required for Computer A:
   $\text{CPU Time}_A = \frac{\text{Clock Cycles}_A}{\text{Clock Rate}_A}$
   $25 \text{ s} = \frac{\text{Clock Cycles}_A}{2\times10^9 \text{ Hz}}$
   $\text{Clock Cycles}_A = 25 \times 2 \times 10^9 = 50 \times 10^9 \text{ cycles}$

2. Calculate the required clock cycles allocated for Computer B:
   $\text{Clock Cycles}_B = 1.2 \times \text{Clock Cycles}_A = 1.2 \times 50 \times 10^9 = 60 \times 10^9 \text{ cycles}$

3. Determine the target clock rate necessary for Computer B to finish processing in exactly 5 seconds:
   $\text{CPU Time}_B = \frac{\text{Clock Cycles}_B}{\text{Clock Rate}_B}$
   $5 \text{ s} = \frac{60\times10^9 \text{ cycles}}{\text{Clock Rate}_B}$
   $\text{Clock Rate}_B = \frac{60\times10^9}{5} = 12 \times 10^9 \text{ Hz} = \boxed{\textbf{12 GHz}}$

2. (*15 points*) General Computer Architecture Concepts

Please provide concise answers to the following foundational architecture questions:

(a) Draw a functional, high-level block diagram of a modern, general-purpose von Neumann computer architecture. Be sure to label the fundamental components and structural data/control pathways.

Solution: *Students should sketch the standard 5-component model. (1) Input Device data feeds into (2) Memory. Memory shares a bidirectional communication path with the structurally combined (3) Datapath/ALU and (4) Control structural block (which collectively compose the Processor/CPU). Output flows synchronously from Memory to the (5) Output device.*

(b) Explicitly define the "Stored-Program Concept" utilizing only 1 or 2 sentences.

Solution: *The Stored-Program Concept states that executing logic instructions are structurally stored in the exact same physical Memory architecture as raw data values. This allows the computer hardware to natively interpret data arrays as structural programs, permitting the machine to run dynamic software dynamically without physical rewiring.*

(c) State the four primary design principles of structural computer architecture.

Solution:

(a) **Simplicity favors regularity**: *Keeping hardware natively consistent (e.g. 32-bit registers) simplifies control implementation.*

(b) **Smaller is faster**: *A limited cache of registers logically evaluates faster than massive raw memory due to fewer electronics.*

(c) **Make the common case fast**: *Optimize performance for the native structural instructions that execute most frequently.*

(d) **Good design demands good compromises**: *Balancing raw hardware formats (like combining exactly 3 native MIPS formats) provides structured compromise over messy arbitrary lengths.*

(d) Briefly define the difference in memory layout between a **Big-Endian** architecture and a **Little-Endian** architecture.

Solution: *Memory layout indexes multi-byte words differently.* ***Big-Endian*** *structurally stores the physical Most Significant Byte (MSB) at the absolute lowest memory address location.* ***Little-Endian*** *mathematically stores the Least Significant Byte (LSB) at the absolute lowest memory address location (the Intel/MIPS native default).*

3. (*15 points*) MIPS Architecture & Layout

Answer the following questions regarding the internal mechanics of the MIPS structural layout:

(a) What specifically defines a "Load-Store Architecture" compared to a register-memory architecture?

Solution: *A native Load-Store hardware architecture dictates that mathematical/logical Arithmetic Logic Unit (ALU) operations can physically only process explicitly against hardware registers format. Standard structural variables existing natively in RAM cannot be computationally operated on until they are logically brought inside the CPU explicitly via 'load' commands and pushed out via 'store' commands.*

(b) List the three defined instruction formats (bit layouts) utilized in the MIPS methodology. Provide the structural name along with a single 1-sentence example mapping of its 32-bit layout fields.

Solution:

(a) **R-Type (Register):** `opcode(6) / rs(5) / rt(5) / rd(5) / shamt(5) / funct(6).`

(b) **I-Type (Immediate):** `opcode(6) / rs(5) / rt(5) / immediate(16).`

(c) **J-Type (Jump):** `opcode(6) / addressTarget(26).`

(c) Briefly explain the layout difference between the `.data` section and the `.text` section in a MIPS assembly program source file. Assuming virtual memory begins addressing at `0x00000000`, structurally explain where each of these partitions natively resides in memory space.

Solution: *The `.text` software block designates the structural physical program logic instructions that natively run computationally. The `.data` software block formally initializes global logic and static raw memory allocations mapped out physically before runtime execution. Natively, `.text` logic functionally sits at `0x0040_0000` natively while the structural `.data` section initializes logically at `0x1000_0000`.*

(d) Explain the fundamental difference between Base/Offset Addressing (e.g. `lw $t0, 4($s0)`) and PC-Relative Addressing (e.g. `bne $s0, $s1, Loop`).

Solution: *Base/Offset computationally utilizes the raw memory logic of taking a native integer Immediate (e.g. 4) and dynamically adding it natively against the numerical memory value held securely in the physical Base Register (e.g. $s0). PC-Relative computationally assumes the jump distance relative to the Program Counter logically, determining target jumps dynamically via `PC + (Immediate * 4 bytes)`.*

4. (*20 points*) Assembly Compilation and Disassembly

(a) **Compilation (Assembly → Machine Code):** Convert the MIPS instruction `lw $t1, -4($s3)` physically into its corresponding 32-bit hexadecimal machine code representation. You must show the structural breakdown of the logical instruction format bits, binary to hex mapping, and the exact two's complement integer calculation.

(*Hint: `lw` opcode is `0x23` / `100011`. `$t1` is register 9. `$s3` is register 19.*)

Solution:

(a) Identify logical format: I-Type (`opcode rs rt immediate`)

(b) `lw` opcode: `0x23` → `100011`

(c) `rs` (base structural pointer): `$s3` is register 19 → `10011`

(d) `rt` (targeted hardware loading register): `$t1` is register 9 → `01001`

(e) Compile **-4** correctly into 16-bit physical two's complement integer logic: `0000 0000 0000 0100` → invert: `1111 1111 1111 1011` → add 1: `1111 1111 1111 1100` (`0xFFFC`)

(f) Construct 32-bit field structure natively: `100011_10011_01001_1111111111111100`

(g) Convert cleanly to Hexadecimal: `1000` — `1110` — `0110` — `1001` — `1111` — `1111` — `1111` — `1100`

(h) **0x8E69FFFC**

(b) **Disassembly (Machine Code → Assembly):** Suppose memory contains the data `0xAE0B0008`. Assuming this value physically represents a native MIPS instruction, convert this hexadecimal machine code backward into its raw MIPS assembly language representation. Show the binary field breakdown natively.

(*Hint: `0xAE0B0008` begins with `1010 1110`. Opcode `101011` maps to `sw`.*)

Solution:

(a) Deconstruct from Hex natively into physical binary sequence: `1010_1110_0000_1011_0000_0000_0000_1000`

(b) Extract Opcode natively (6-bits): `101011` → Native `sw` command.

(c) Recognize I-Type format logically mapping into `sw rt, imm(rs)`

(d) Extract `rs` natively (next 5 bits): `10000` → target register 16 → `$s0`

(e) Extract `rt` natively (next 5 bits): `01011` → target register 11 → `$t3`

(f) Extract `immediate` natively (last 16 bits): `0000 0000 0000 1000` → absolute 8.

(g) Combine into structured assembly program instruction sequence.

(h) **sw $t3, 8($s0)**

5. (*15 points*) **Translating a C array manipulation (Leaf Procedure)**

Compile the following C 'for' loop logic natively into structural MIPS assembly. Assume `$s0 = &arr[0]` natively holds the base address of the array natively, and `$s1 = size = 10`. Assume the C iterations variables `sum`, `pos`, and `neg` structurally map to hardware registers `$t0`, `$t1`, and `$t2` respectively and have an initial physical state of 0. Finally, utilize `$t3` as the loop iterative variable `i` native state.

Note: This is an internal *Leaf* operation block! Do <u>not</u> manipulate the stack pointer (`$sp`) physically for this specific implementation. Simply map the functional data logic into optimal assembly.

```
// Base integer initializations already handled
for (i = 0; i < size; i++) {
    sum += arr[i];
    if (arr[i] > 0)
        pos += arr[i];
    if (arr[i] < 0)
        neg += arr[i];
}
```

Write your physical MIPS Assembly natively below:

Solution:

```
# Registers already set: $t0=sum, $t1=pos, $t2=neg, $s0=arr, $s1=size
    add $t3, $0, $0    # Initialize i = 0

FOR_LOOP:
    slt $t4, $t3, $s1 # Check condition: $t4 = (i < size) natively
    beq $t4, $0, DONE # If logic equates to 0, break loop sequentially

    # Calculate array offset for 'arr[i]'
    sll $t5, $t3, 2    # Compute word length offset: $t5 = i * 4 bytes
    add $t5, $s0, $t5 # Build physical destination: $t5 = arr + offset
    lw  $t6, 0($t5)    # Load physical variable: $t6 = arr[i]

    # Execute native summation logic
    add $t0, $t0, $t6 # Implement functionally: sum += arr[i]

    # Native evaluation block -> if (arr[i] > 0)
    slt $t7, $0, $t6   # $t7 = (0 < arr[i])
    beq $t7, $0, CHECK_NEG
    add $t1, $t1, $t6 # Implement functionally: pos += arr[i]

CHECK_NEG:
    # Native evaluation block -> if (arr[i] < 0)
    slt $t7, $t6, $0   # $t7 = (arr[i] < 0)
    beq $t7, $0, NEXT_ITER
    add $t2, $t2, $t6 # Implement functionally: neg += arr[i]

NEXT_ITER:
    addi $t3, $t3, 1   # Standard step: i++
    j FOR_LOOP         # Loop execution sequence back

DONE:
    # Program executes cleanly
```

6. (*20 points*) **Translating Nested Procedures (Stack Pointer Management)**

Translate the following nested recursive C factorial procedure physically into native MIPS assembly language. Ensure extreme caution regarding the MIPS Register Conventions! You must demonstrate accurate programmatic pushes and pops to the hardware Stack via manipulating the structural `$sp` vector and safely restoring `$ra` values logically. Assume integer `n` is physically passed into your subprogram natively via `$a0`, and standard results are returned synchronously onto `$v0`.

```
int fact(int n)
{
    if (n < 1)
        return (1);
    else
        return (n * fact(n - 1));
}
```

Write your physical MIPS Assembly natively below:

Solution:

```
fact:
    # Nested procedures require structural tracking! Save contextual variables.
    addi $sp, $sp, -8    # Scale hardware Stack Pointer native limit.
    sw   $ra, 4($sp)     # Push native structurally caller Return Address physically.
    sw   $a0, 0($sp)     # Push local mathematical value (n) hardware array.

    # Base native evaluation limit
    slti $t0, $a0, 1     # Mathematical logic: $t0 = (n < 1)
    beq  $t0, $0, RECURSE # Standard recursive loop condition execution.

    # Internal BASE RETURN conditional state.
    addi $v0, $0, 1      # Set standard return natively.
    addi $sp, $sp, 8     # Pop structural stack array elements off hardware safely.
    jr   $ra             # Bounce logical state array back to caller execution natively.

RECURSE:
    # Recursive Execution Sequence Array Limit calculation.
    addi $a0, $a0, -1    # Computationally decrement mathematically evaluating function parame
    jal  fact            # Nested logical JAL inherently crushes $ra state hardware.

    # Recover state values logically correctly back off structural array limits.
    lw   $a0, 0($sp)     # Restore $a0 logical state hardware array context.
    lw   $ra, 4($sp)     # Safely resurrect nested Return Address limit context natively.
    addi $sp, $sp, 8     # Return Memory limit array bounds natively onto the Stack natively.

    # Conclude math payload operation natively.
    mul  $v0, $a0, $v0   # $v0 = n * fact(n-1) natively
    jr   $ra             # Complete native internal programmatic closure limits natively.
```

Survey Question (2 Extra Credit Points):

What did you think of this functional Mid-Term Exam? Were you mathematically prepared for the kinds of structural evaluations required?

Scrap Page

(please do not remove this page from the test packet)

Scrap Page

(please do not remove this page from the test packet)

MIPS Reference Data Card ("Green Card") 1. Pull along perforation to separate card 2. Fold bottom side (columns 3 and 4) together

MIPS Reference Data

①

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	0 / 20_{hex}
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		0 / 21_{hex}
And	and	R	R[rd] = R[rs] & R[rt]		0 / 24_{hex}
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4)	4_{hex}
Branch On Not Equal	bne	I	if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4)	5_{hex}
Jump	j	J	PC=JumpAddr	(5)	2_{hex}
Jump And Link	jal	J	R[31]=PC+8;PC=JumpAddr	(5)	3_{hex}
Jump Register	jr	R	PC=R[rs]		0 / 08_{hex}
Load Byte Unsigned	lbu	I	R[rt]={24'b0,M[R[rs] +SignExtImm](7:0)}	(2)	24_{hex}
Load Halfword Unsigned	lhu	I	R[rt]={16'b0,M[R[rs] +SignExtImm](15:0)}	(2)	25_{hex}
Load Linked	ll	I	R[rt] = M[R[rs]+SignExtImm]	(2,7)	30_{hex}
Load Upper Imm.	lui	I	R[rt] = {imm, 16'b0}		f_{hex}
Load Word	lw	I	R[rt] = M[R[rs]+SignExtImm]	(2)	23_{hex}
Nor	nor	R	R[rd] = ~ (R[rs] | R[rt])		0 / 27_{hex}
Or	or	R	R[rd] = R[rs] | R[rt]		0 / 25_{hex}
Or Immediate	ori	I	R[rt] = R[rs] | ZeroExtImm	(3)	d_{hex}
Set Less Than	slt	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0		0 / $2a_{hex}$
Set Less Than Imm.	slti	I	R[rt] = (R[rs] < SignExtImm)? 1 : 0	(2)	a_{hex}
Set Less Than Imm. Unsigned	sltiu	I	R[rt] = (R[rs] < SignExtImm) ? 1 : 0	(2,6)	b_{hex}
Set Less Than Unsig.	sltu	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0	(6)	0 / $2b_{hex}$
Shift Left Logical	sll	R	R[rd] = R[rt] << shamt		0 / 00_{hex}
Shift Right Logical	srl	R	R[rd] = R[rt] >> shamt		0 / 02_{hex}
Store Byte	sb	I	M[R[rs]+SignExtImm](7:0) = R[rt](7:0)	(2)	28_{hex}
Store Conditional	sc	I	M[R[rs]+SignExtImm] = R[rt]; R[rt] = (*atomic*) ? 1 : 0	(2,7)	38_{hex}
Store Halfword	sh	I	M[R[rs]+SignExtImm](15:0) = R[rt](15:0)	(2)	29_{hex}
Store Word	sw	I	M[R[rs]+SignExtImm] = R[rt]	(2)	$2b_{hex}$
Subtract	sub	R	R[rd] = R[rs] - R[rt]	(1)	0 / 22_{hex}
Subtract Unsigned	subu	R	R[rd] = R[rs] - R[rt]		0 / 23_{hex}

(1) May cause overflow exception
(2) SignExtImm = { 16{immediate[15]}, immediate }
(3) ZeroExtImm = { 16{1b'0}, immediate }
(4) BranchAddr = { 14{immediate[15]}, immediate, 2'b0 }
(5) JumpAddr = { PC+4[31:28], address, 2'b0 }
(6) Operands considered unsigned numbers (vs. 2's comp.)
(7) Atomic test&set pair; R[rt] = 1 if pair atomic, 0 if not atomic

BASIC INSTRUCTION FORMATS

R	opcode	rs	rt	rd	shamt	funct
	31–26	25–21	20–16	15–11	10–6	5–0
I	opcode	rs	rt	immediate		
	31–26	25–21	20–16	15–0		
J	opcode	address				
	31–26	25–0				

ARITHMETIC CORE INSTRUCTION SET

②

NAME, MNEMONIC		FOR-MAT	OPERATION		OPCODE / FMT /FT / FUNCT (Hex)
Branch On FP True	bc1t	FI	if(FPcond)PC=PC+4+BranchAddr	(4)	11/8/1/--
Branch On FP False	bc1f	FI	if(!FPcond)PC=PC+4+BranchAddr	(4)	11/8/0/--
Divide	div	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]		0/--/--/1a
Divide Unsigned	divu	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]	(6)	0/--/--/1b
FP Add Single	add.s	FR	F[fd]= F[fs] + F[ft]		11/10/--/0
FP Add Double	add.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} + {F[ft],F[ft+1]}		11/11/--/0
FP Compare Single	c.*x*.s*	FR	FPcond = (F[fs] *op* F[ft]) ? 1 : 0		11/10/--/*y*
FP Compare Double	c.*x*.d*	FR	FPcond = ({F[fs],F[fs+1]} *op* {F[ft],F[ft+1]}) ? 1 : 0		11/11/--/*y*

* (*x* is eq, lt, or le) (*op* is ==, <, or <=) (*y* is 32, 3c, or 3e)

FP Divide Single	div.s	FR	F[fd] = F[fs] / F[ft]		11/10/--/3
FP Divide Double	div.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} / {F[ft],F[ft+1]}		11/11/--/3
FP Multiply Single	mul.s	FR	F[fd] = F[fs] * F[ft]		11/10/--/2
FP Multiply Double	mul.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} * {F[ft],F[ft+1]}		11/11/--/2
FP Subtract Single	sub.s	FR	F[fd]=F[fs] - F[ft]		11/10/--/1
FP Subtract Double	sub.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} - {F[ft],F[ft+1]}		11/11/--/1
Load FP Single	lwc1	I	F[rt]=M[R[rs]+SignExtImm]	(2)	31/--/--/--
Load FP Double	ldc1	I	F[rt]=M[R[rs]+SignExtImm]; F[rt+1]=M[R[rs]+SignExtImm+4]	(2)	35/--/--/--
Move From Hi	mfhi	R	R[rd] = Hi		0 /--/--/10
Move From Lo	mflo	R	R[rd] = Lo		0 /--/--/12
Move From Control	mfc0	R	R[rd] = CR[rs]		10 /0/--/0
Multiply	mult	R	{Hi,Lo} = R[rs] * R[rt]		0/--/--/18
Multiply Unsigned	multu	R	{Hi,Lo} = R[rs] * R[rt]	(6)	0/--/--/19
Shift Right Arith.	sra	R	R[rd] = R[rt] >>> shamt		0/--/--/3
Store FP Single	swc1	I	M[R[rs]+SignExtImm] = F[rt]	(2)	39/--/--/--
Store FP Double	sdc1	I	M[R[rs]+SignExtImm] = F[rt]; M[R[rs]+SignExtImm+4] = F[rt+1]	(2)	3d/--/--/--

FLOATING-POINT INSTRUCTION FORMATS

FR	opcode	fmt	ft	fs	fd	funct
	31–26	25–21	20–16	15–11	10–6	5–0
FI	opcode	fmt	ft	immediate		
	31–26	25–21	20–16	15–0		

PSEUDOINSTRUCTION SET

NAME	MNEMONIC	OPERATION
Branch Less Than	blt	if(R[rs]<R[rt]) PC = Label
Branch Greater Than	bgt	if(R[rs]>R[rt]) PC = Label
Branch Less Than or Equal	ble	if(R[rs]<=R[rt]) PC = Label
Branch Greater Than or Equal	bge	if(R[rs]>=R[rt]) PC = Label
Load Immediate	li	R[rd] = immediate
Move	move	R[rd] = R[rs]

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
$zero	0	The Constant Value 0	N.A.
$at	1	Assembler Temporary	No
$v0-$v1	2-3	Values for Function Results and Expression Evaluation	No
$a0-$a3	4-7	Arguments	No
$t0-$t7	8-15	Temporaries	No
$s0-$s7	16-23	Saved Temporaries	Yes
$t8-$t9	24-25	Temporaries	No
$k0-$k1	26-27	Reserved for OS Kernel	No
$gp	28	Global Pointer	Yes
$sp	29	Stack Pointer	Yes
$fp	30	Frame Pointer	Yes
$ra	31	Return Address	Yes

Copyright 2009 by Elsevier, Inc., All rights reserved. From Patterson and Hennessy, *Computer Organization and Design,* 4th ed.

OPCODES, BASE CONVERSION, ASCII SYMBOLS

③

MIPS opcode (31:26)	(1) MIPS funct (5:0)	(2) MIPS funct (5:0)	Binary	Decimal	Hexadecimal	ASCII Character	Decimal	Hexadecimal	ASCII Character
(1)	sll	add.*f*	00 0000	0	0	NUL	64	40	@
		sub.*f*	00 0001	1	1	SOH	65	41	A
j	srl	mul.*f*	00 0010	2	2	STX	66	42	B
jal	sra	div.*f*	00 0011	3	3	ETX	67	43	C
beq	sllv	sqrt.*f*	00 0100	4	4	EOT	68	44	D
bne		abs.*f*	00 0101	5	5	ENQ	69	45	E
blez	srlv	mov.*f*	00 0110	6	6	ACK	70	46	F
bgtz	srav	neg.*f*	00 0111	7	7	BEL	71	47	G
addi	jr		00 1000	8	8	BS	72	48	H
addiu	jalr		00 1001	9	9	HT	73	49	I
slti	movz		00 1010	10	a	LF	74	4a	J
sltiu	movn		00 1011	11	b	VT	75	4b	K
andi	syscall	round.w.*f*	00 1100	12	c	FF	76	4c	L
ori	break	trunc.w.*f*	00 1101	13	d	CR	77	4d	M
xori		ceil.w.*f*	00 1110	14	e	SO	78	4e	N
lui	sync	floor.w.*f*	00 1111	15	f	SI	79	4f	O
	mfhi		01 0000	16	10	DLE	80	50	P
(2)	mthi		01 0001	17	11	DC1	81	51	Q
	mflo	movz.*f*	01 0010	18	12	DC2	82	52	R
	mtlo	movn.*f*	01 0011	19	13	DC3	83	53	S
			01 0100	20	14	DC4	84	54	T
			01 0101	21	15	NAK	85	55	U
			01 0110	22	16	SYN	86	56	V
			01 0111	23	17	ETB	87	57	W
	mult		01 1000	24	18	CAN	88	58	X
	multu		01 1001	25	19	EM	89	59	Y
	div		01 1010	26	1a	SUB	90	5a	Z
	divu		01 1011	27	1b	ESC	91	5b	[
			01 1100	28	1c	FS	92	5c	\
			01 1101	29	1d	GS	93	5d	]
			01 1110	30	1e	RS	94	5e	^
			01 1111	31	1f	US	95	5f	_
lb	add	cvt.s.*f*	10 0000	32	20	Space	96	60	`
lh	addu	cvt.d.*f*	10 0001	33	21	!	97	61	a
lwl	sub		10 0010	34	22	"	98	62	b
lw	subu		10 0011	35	23	#	99	63	c
lbu	and	cvt.w.*f*	10 0100	36	24	$	100	64	d
lhu	or		10 0101	37	25	%	101	65	e
lwr	xor		10 0110	38	26	&	102	66	f
	nor		10 0111	39	27	'	103	67	g
sb			10 1000	40	28	(	104	68	h
sh			10 1001	41	29	)	105	69	i
swl	slt		10 1010	42	2a	*	106	6a	j
sw	sltu		10 1011	43	2b	+	107	6b	k
			10 1100	44	2c	,	108	6c	l
			10 1101	45	2d	-	109	6d	m
swr			10 1110	46	2e	.	110	6e	n
cache			10 1111	47	2f	/	111	6f	o
ll	tge	c.f.*f*	11 0000	48	30	0	112	70	p
lwc1	tgeu	c.un.*f*	11 0001	49	31	1	113	71	q
lwc2	tlt	c.eq.*f*	11 0010	50	32	2	114	72	r
pref	tltu	c.ueq.*f*	11 0011	51	33	3	115	73	s
	teq	c.olt.*f*	11 0100	52	34	4	116	74	t
ldc1		c.ult.*f*	11 0101	53	35	5	117	75	u
ldc2	tne	c.ole.*f*	11 0110	54	36	6	118	76	v
		c.ule.*f*	11 0111	55	37	7	119	77	w
sc		c.sf.*f*	11 1000	56	38	8	120	78	x
swc1		c.ngle.*f*	11 1001	57	39	9	121	79	y
swc2		c.seq.*f*	11 1010	58	3a	:	122	7a	z
		c.ngl.*f*	11 1011	59	3b	;	123	7b	{
		c.lt.*f*	11 1100	60	3c	<	124	7c	|
sdc1		c.nge.*f*	11 1101	61	3d	=	125	7d	}
sdc2		c.le.*f*	11 1110	62	3e	>	126	7e	~
		c.ngt.*f*	11 1111	63	3f	?	127	7f	DEL

(1) opcode(31:26) == 0

(2) opcode(31:26) == 17_{ten} (11_{hex}); if fmt(25:21)==16_{ten} (10_{hex}) *f* = s (single); if fmt(25:21)==17_{ten} (11_{hex}) *f* = d (double)

IEEE 754 FLOATING-POINT STANDARD

④

$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

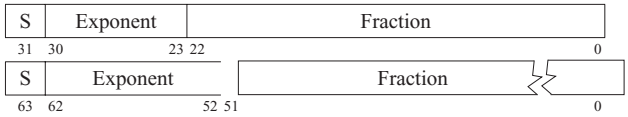
where Single Precision Bias = 127, Double Precision Bias = 1023.

IEEE 754 Symbols

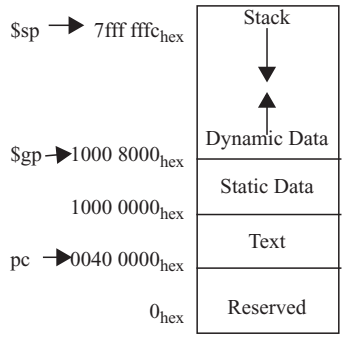
Exponent	Fraction	Object
0	0	± 0
0	≠0	± Denorm
1 to MAX - 1	anything	± Fl. Pt. Num.
MAX	0	±∞
MAX	≠0	NaN

S.P. MAX = 255, D.P. MAX = 2047

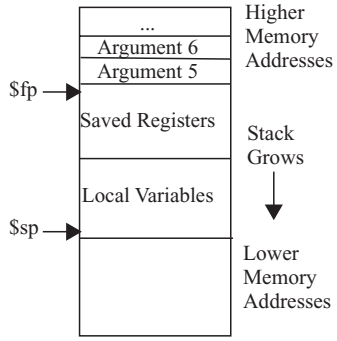
IEEE Single Precision and Double Precision Formats:



MEMORY ALLOCATION



STACK FRAME

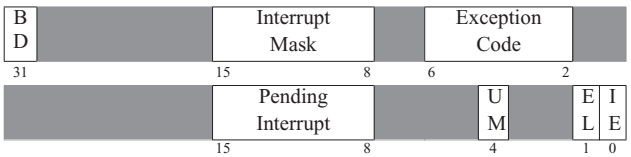


DATA ALIGNMENT

Double Word							
Word				Word			
Halfword		Halfword		Halfword		Halfword	
Byte	Byte	Byte	Byte	Byte	Byte	Byte	Byte
0	1	2	3	4	5	6	7

Value of three least significant bits of byte address (Big Endian)

EXCEPTION CONTROL REGISTERS: CAUSE AND STATUS



BD = Branch Delay, UM = User Mode, EL = Exception Level, IE =Interrupt Enable

EXCEPTION CODES

Number	Name	Cause of Exception	Number	Name	Cause of Exception
0	Int	Interrupt (hardware)	9	Bp	Breakpoint Exception
4	AdEL	Address Error Exception (load or instruction fetch)	10	RI	Reserved Instruction Exception
5	AdES	Address Error Exception (store)	11	CpU	Coprocessor Unimplemented
6	IBE	Bus Error on Instruction Fetch	12	Ov	Arithmetic Overflow Exception
7	DBE	Bus Error on Load or Store	13	Tr	Trap
8	Sys	Syscall Exception	15	FPE	Floating Point Exception

SIZE PREFIXES (10^x for Disk, Communication; 2^x for Memory)

SIZE	PREFIX	SIZE	PREFIX	SIZE	PREFIX	SIZE	PREFIX
10^3, 2^{10}	Kilo-	10^{15}, 2^{50}	Peta-	10^{-3}	milli-	10^{-15}	femto-
10^6, 2^{20}	Mega-	10^{18}, 2^{60}	Exa-	10^{-6}	micro-	10^{-18}	atto-
10^9, 2^{30}	Giga-	10^{21}, 2^{70}	Zetta-	10^{-9}	nano-	10^{-21}	zepto-
10^{12}, 2^{40}	Tera-	10^{24}, 2^{80}	Yotta-	10^{-12}	pico-	10^{-24}	yocto-

The symbol for each prefix is just its first letter, except μ is used for micro.

Copyright 2009 by Elsevier, Inc., All rights reserved. From Patterson and Hennessy, *Computer Organization and Design,* 4th ed.

MIPS Reference Data Card ("Green Card") 1. Pull along perforation to separate card 2. Fold bottom side (columns 3 and 4) together